Unidad II: Colas de prioridad y montículos

Estructura de Datos II

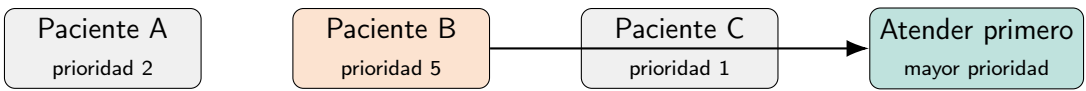
Material teórico para clase

Universidad Autónoma de Aguascalientes

Idea central

Pregunta guía

Una cola normal atiende por orden de llegada.
Una cola de prioridad atiende según la importancia de cada elemento.



No siempre sale el primero que entró; sale el más importante.

Aplicaciones típicas

Sistemas

- Procesos del sistema operativo.
- Atención de eventos.
- Simuladores.
- Impresoras compartidas.

Algoritmos

- Dijkstra.
- Prim.
- Huffman.
- Planificación de tareas.

Punto clave

La estructura debe responder rápido: **¿quién tiene mayor o menor prioridad?**

Operaciones de una cola de prioridad

Operación	Descripción
insertar	Agrega un nuevo elemento con una prioridad.
consultar	Observa el elemento de mayor prioridad sin eliminarlo.
extraer	Elimina y devuelve el elemento de mayor prioridad.
está vacía	Indica si no hay elementos disponibles.
tamaño	Indica cuántos elementos contiene.

Convención de clase

En un **Max-Heap**, el número mayor representa mayor prioridad.
En un **Min-Heap**, el número menor representa mayor prioridad.

Implementaciones posibles

Implementación	Insertar	Consultar máximo	Extraer máximo
Arreglo desordenado	Rápido	Lento	Lento
Arreglo ordenado	Lento	Rápido	Rápido
Lista enlazada	Variable	Variable	Variable
Montículo binario	Rápido	Muy rápido	Rápido

Idea para explicar

El montículo es un punto medio eficiente: no ordena todo, solo mantiene arriba al elemento prioritario.

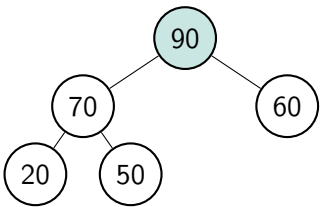
¿Qué es un montículo binario?

Definición informal

Un montículo binario es un árbol binario casi completo que cumple una propiedad de prioridad entre padre e hijos.

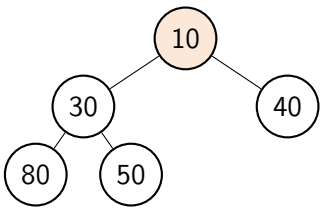
Max-Heap

Cada padre es mayor o igual que sus hijos.



Min-Heap

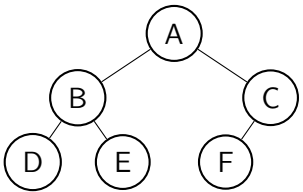
Cada padre es menor o igual que sus hijos.



Dos propiedades del montículo

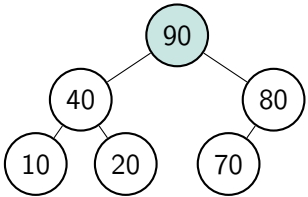
1. Forma

El árbol debe estar completo de izquierda a derecha, excepto quizá el último nivel.

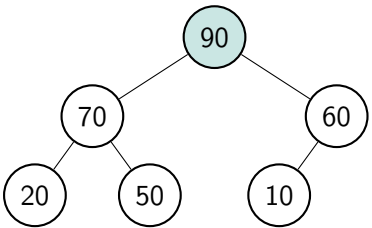


2. Orden parcial

No exige que todo esté ordenado. Solo exige relación padre-hijo.



Representación con vector



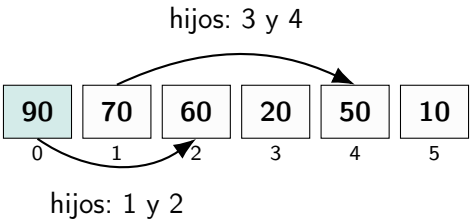
90	70	60	20	50	10
0	1	2	3	4	5

Fórmulas usando índice desde 0

Para un nodo en la posición i:

- Padre: $\left\lfloor \frac{i-1}{2} \right\rfloor$
- Hijo izquierdo: $2i + 1$
- Hijo derecho: $2i + 2$

Ventaja del vector



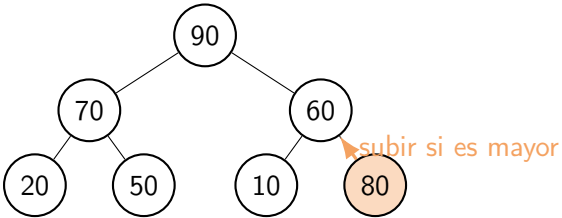
Mensaje para los alumnos

Aunque lo dibujamos como árbol, normalmente se guarda en un vector.
No se necesitan punteros para moverse entre padre e hijos: se usan fórmulas.

Operación: insertar en Max-Heap

Idea

El nuevo elemento se coloca al final para conservar la forma de árbol completo.
Luego sube mientras tenga mayor prioridad que su padre.



Pseudocódigo: insertar

Insertar(elemento)

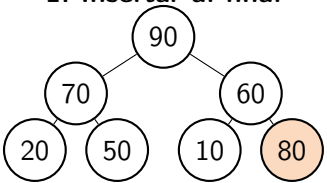
1. Agregar el elemento al final del vector.
2. Sea i la posición del nuevo elemento.
3. Mientras i no sea la raíz y el elemento tenga mayor prioridad que su padre:
   1. Intercambiar el elemento con su padre.
   2. Actualizar i a la posición del padre.
4. Terminar cuando se restaure la propiedad de montículo.

Nombre común

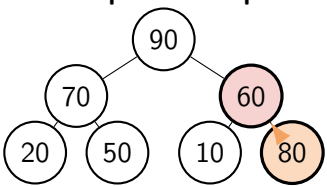
A esta subida se le suele llamar **sift-up**, **bubble-up** o **flotar**.

Secuencia gráfica: insertar 80

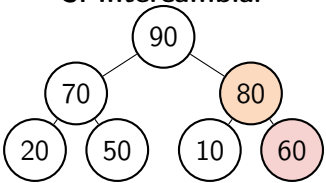
1. Insertar al final



2. Comparar con padre

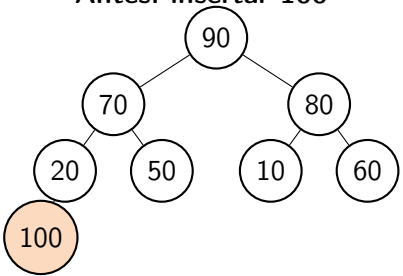


3. Intercambiar

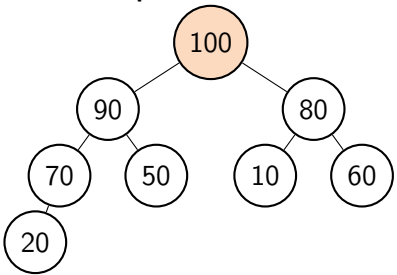


Insertar: caso donde sube más de un nivel

Antes: insertar 100



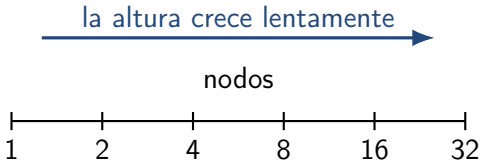
Después de subir



Costo de insertar

Análisis

En el peor caso, el nuevo elemento sube desde la última posición hasta la raíz.



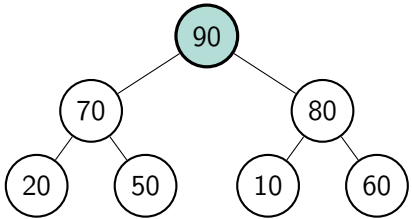
Resultado

La altura del montículo es proporcional a $\log_2(n)$, por eso insertar cuesta $O(\log n)$.

Operación: consultar mayor prioridad

Max-Heap

El elemento con mayor prioridad siempre está en la raíz.



Consultar = 90

Costo

Consultar la raíz cuesta $O(1)$.

Operación: extraer máximo

Problema

Si quitamos la raíz directamente, se rompe la forma del árbol.

Solución

1. Guardar la raíz para devolverla.
2. Mover el último elemento a la raíz.
3. Eliminar la última posición del vector.
4. Bajar el elemento de la raíz hasta restaurar la propiedad del montículo.

Pseudocódigo: extraer máximo

ExtraerMax()

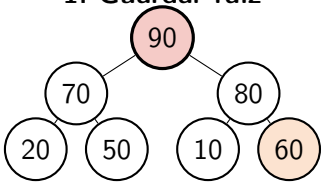
1. Si el montículo está vacío, reportar error o vacío.
2. Guardar el valor de la raíz como respuesta.
3. Mover el último elemento a la raíz.
4. Reducir el tamaño del montículo en uno.
5. Aplicar **sift-down** desde la raíz.
6. Devolver la respuesta guardada.

Nombre común

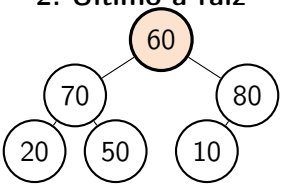
A esta bajada se le llama **sift-down**, **sink** o **hundir**.

Secuencia gráfica: extraer máximo

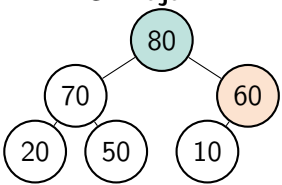
1. Guardar raíz



2. Último a raíz



3. Bajar



Pseudocódigo: sift-down

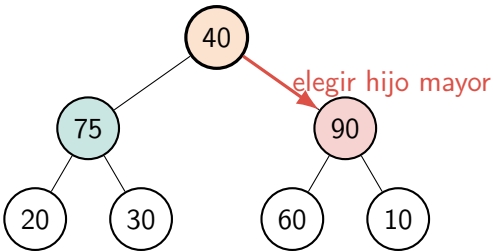
SiftDown(posición)

1. Mientras la posición tenga al menos un hijo:
   1. Seleccionar el hijo con mayor prioridad.
   2. Si el padre ya tiene mayor o igual prioridad que ese hijo, detener.
   3. Si no, intercambiar padre e hijo.
   4. Continuar desde la nueva posición del elemento que bajó.

Idea visual

El elemento que quedó arriba se compara con sus hijos y va bajando por el camino del hijo más fuerte.

Extraer: decidir con qué hijo intercambiar



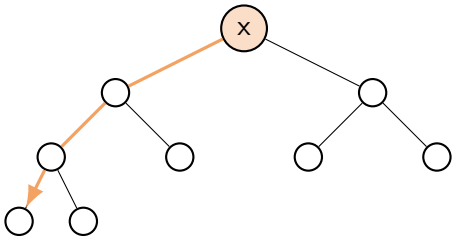
Regla

En Max-Heap se intercambia con el hijo mayor. En Min-Heap se intercambia con el hijo menor.

Costo de extraer

Análisis

El elemento movido a la raíz puede bajar hasta el último nivel.



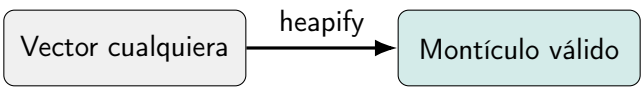
Resultado

Extraer el máximo o mínimo cuesta $O(\log n)$.

Construir un montículo

Dos formas

1. Insertar elementos uno por uno.
2. Reorganizar un vector existente usando heapify desde abajo hacia arriba.



Pseudocódigo: construir por inserciones

ConstruirPorInserciones(lista)

1. Crear un montículo vacío.
2. Para cada elemento de la lista:
   1. Insertar el elemento en el montículo.
3. Devolver el montículo.

Costo

Como cada inserción puede costar $O(\log n)$, construir así cuesta $O(n \log n)$.

Pseudocódigo: construir con heapify

ConstruirHeap(vector)

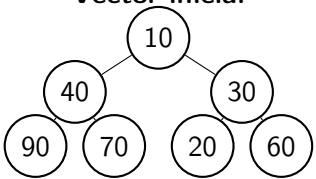
1. Tomar el vector completo como si fuera un árbol completo.
2. Ubicarse en el último nodo que tiene hijos.
3. Desde ese nodo hasta la raíz:
   1. Aplicar **sift-down** en la posición actual.
   2. Avanzar una posición hacia atrás.
4. Al terminar, el vector cumple la propiedad de montículo.

Costo

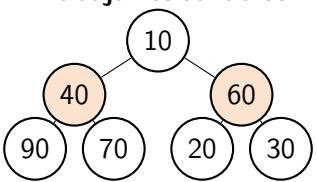
Construir con heapify cuesta $O(n)$.

Secuencia gráfica: heapify desde abajo

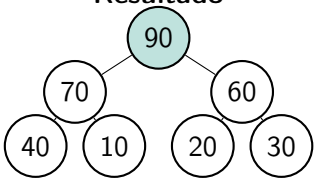
Vector inicial



Trabajar subárboles



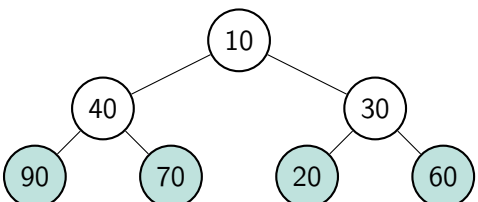
Resultado



Heapify: ¿por qué desde abajo?

Razón visual

Los nodos hoja ya son montículos de tamaño 1. No hay nada que corregir en ellos.



Las hojas ya están correctas. Se corrigen primero los padres de hojas.

Ordenación por montículos: HeapSort

Idea

Si siempre podemos extraer el máximo, entonces podemos ir colocando los máximos al final del vector.

90	70	60	40	10	20	30

Montículo dentro del mismo vector

No usa estructura extra grande

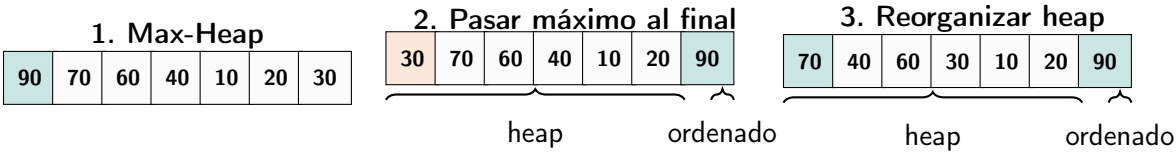
HeapSort trabaja principalmente intercambiando elementos dentro del mismo arreglo.

Pseudocódigo: HeapSort

HeapSort(vector)

1. Construir un Max-Heap con todos los elementos.
2. Mientras el tamaño del heap sea mayor que 1:
   1. Intercambiar la raíz con el último elemento del heap.
   2. Reducir el tamaño lógico del heap en uno.
   3. Aplicar **sift-down** en la raíz.
3. El vector queda ordenado de menor a mayor.

Secuencia gráfica: HeapSort



Repetir

En cada vuelta, el mayor del heap se congela al final del vector.

Complejidad de HeapSort

Etapa	Costo
Construir heap	$O(n)$
Extraer máximo n veces	$O(n \log n)$
Costo total	$O(n \log n)$
Memoria adicional	$O(1)$

Comentario didáctico

HeapSort no necesita ordenar todo el árbol visualmente: solo mantiene el heap activo y una zona final ya ordenada.

Max-Heap vs. Min-Heap

Max-Heap

- La raíz es el mayor.
- Sirve para extraer máximos.
- Útil si la prioridad alta se representa con número grande.

Min-Heap

- La raíz es el menor.
- Sirve para extraer mínimos.
- Útil en Dijkstra, eventos por tiempo, costos mínimos.

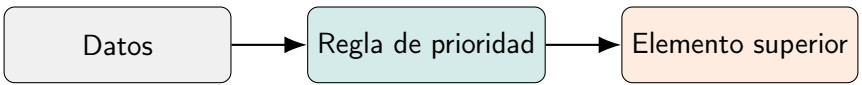
Max-Heap: 90 arriba

Min-Heap: 10 arriba

Uso conceptual de `priority_queue`

Idea

En C++, `priority_queue` es un adaptador de contenedor. Internamente mantiene una estructura tipo montículo.



Para clase

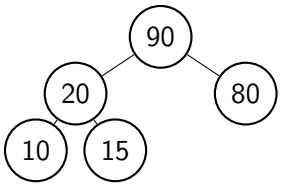
Aquí conviene explicar el concepto, no memorizar sintaxis. La sintaxis se puede practicar después.

Errores comunes

1. Creer que un heap es un árbol binario de búsqueda.
2. Pensar que el vector completo queda ordenado después de cada inserción.
3. Intercambiar con cualquier hijo en **sift-down**; debe elegirse el hijo de mayor prioridad.
4. Confundir la forma completa del árbol con la propiedad de prioridad.
5. Olvidar reducir el tamaño lógico del heap en HeapSort.

Heap no es BST

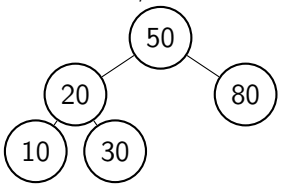
Heap



Padre mayor que hijos.

BST

Izquierda menor, derecha mayor.



Diferencia clave

En un heap no se busca como en BST. El objetivo principal es acceder rápido al elemento prioritario.

Resumen de complejidades

Operación	Costo	Motivo
Consultar raíz	$O(1)$	Está en la primera posición.
Insertar	$O(\log n)$	Puede subir hasta la raíz.
Extraer máximo/mínimo	$O(\log n)$	Puede bajar hasta una hoja.
Construir heap con heapify	$O(n)$	Reorganiza desde abajo.
HeapSort	$O(n \log n)$	Repite extracciones.